

Deploying the [NammaAUTOSAR stack](#) on a Raspberry Pi 5 running Linux is an excellent learning path.

Because NammaAUTOSAR is a classic AUTOSAR platform designed to abstract target microcontrollers, you do not flash it bare-metal on the Pi 5. Instead, you run it inside the **Raspberry Pi 5's Linux terminal as a POSIX application**. The Pi 5's fast 64-bit architecture makes compiling and simulating the cyclic RTE and BSW tasks instantaneous. [1, 2]

Below is the complete step-by-step developer's guide to deploying the stack on your Raspberry Pi 5 to learn RTE and BSW configurations.

Step 1: Install Prerequisites on Raspberry Pi 5

Open the terminal on your Raspberry Pi 5 (running standard Raspberry Pi OS or Ubuntu) and install the standard GNU compilation toolchains and Python dependencies required by the NammaAUTOSAR build tools: [3]

```
sudo apt update
sudo apt install -y git build-essential cmake python3 python3-pip python3-tk
```

Step 2: Clone NammaAUTOSAR with Submodules

NammaAUTOSAR architecture splits its BSW modules (like Os, Com, and Dio) into separate internal repositories. You must clone the repository with the `--recursive` flag to capture these components: [1]

```
# Clone the repository along with all its active submodules
git clone --recursive https://github.com
cd NammaAUTOSAR
```

(Note: As specified in the NammaAUTOSAR GitHub documentation, stick to the main project branch for classic architectural learning before moving to their newer Zephyr-RTOS tracking codeports). [1, 4]

Step 3: Run the AUTOSAR Configuration GUI

AUTOSAR code is configuration-driven. Before code execution, tools parse configuration data to generate the RTE and structural BSW mappings. NammaAUTOSAR includes a custom Python-based GUI tool for this: [2, 5]

```
# Start the architectural tool inside your Pi desktop terminal
python3 tools/autosar-gui.py
```

What to look for in the GUI:

-
- Navigate to the **OS (Operating System)** tab to inspect tasks, alarms, and priority structures.
- Open the **RTE** panel to inspect how Application Software Component (SWC) ports are mapped to internal BSW signals.
- Click **Generate** (or use the save function) to construct the generated .c and .h runtime files. [6]
-

Step 4: Map Hardware to the Linux Target

When executing on the Pi 5 Linux environment, the low-level **MCAL (Microcontroller Abstraction Layer)** needs to be swapped out for POSIX simulator calls.

1. Look into the /src/bsw/mcal/ or target directories inside the repository.
2. The code directs hardware timer interrupts to use standard POSIX timer_create loops or loop loops inside a master Linux thread.
3. Peripheral drivers (like the **Dio - Digital Input/Output** stack) map directly to terminal outputs (printf) or standard /sys/class/gpio mock arrays inside Linux rather than physical raw registers.

Step 5: Compile and Run the Execution Loop

Once configurations are generated, compile the stack utilizing the provided Makefiles or CMake parameters targeting the host system:

```
# Clean previous builds and compile for the POSIX/Linux target platform
make clean
make target=posix
```

Run the compiled executable:

```
./build/namma_autosar_sim
```

How to Use This Setup to Learn RTE & BSW

Once the simulator is running on your Pi 5 terminal, modify the source code to see how the platform functions:

1. Trace the BSW Scheduling Mechanism

Open the Os (Operating System) source files. Look for the cyclic task loops. You will see how the OS schedules the **BSW Main Functions** (e.g., Com_MainFunction_Tx(), Can_MainFunction_Read()). Observe how they run at rigid intervals (e.g., every 10ms or 20ms) to update network and diagnostic layers.

2. Trace the RTE Data Flow

Create a new dummy port mapping in the GUI or manually track the Rte.c files. Trace exactly what happens when an Application Software Component executes Rte_Write_Port_Signal(). You will see the RTE act as a post office:

-
- If the data is meant for another application on the *same* system, the RTE moves it into a shared memory variable.
- If the data is meant for an *external* system, the RTE passes it down to the **BSW Com module**, which packs it into a signal matrix for transmission.
-

Would you like to walk through the exact steps to **manually modify an application Runnable** within the NammaAUTOSAR directory, or would you prefer to break down how its **OS task scheduler** functions on top of Linux?

[1] <https://github.com>

[2] <https://github.com>

[3] <https://medium.com>

[4] <https://github.com>

[5] <https://maaz.global>

[6] <https://medium.com>

